

AI 如何辅助软件工程？

成为 AI 时代的技术经理，与其被动被抛弃，不如主动迎接

需求衔接

如何用 AI 整理需求？

- 如何用 Markdown 编写需求？
 - 如何切分模块？颗粒度多大？
 - 如何表达表单字段？
 - 如何组合原型设计？
 - 如何表达业务规则？
- Markdown 需求的模版长什么样？
 - 表达新建需求的模版？
 - 表达变更需求的模版？

开发实现

如何获得高质量的代码？

- 如何让 AI 听话？
- 如何让 AI 调用外部工具？
- 技术规格如何编写？
- 打样工程：让 AI 抄作业
- 如何多任务同步开发？
- 如何可视化技术规格用于评审？
- 如何自动化触发代码提交、合并的问题？

测试验收

如何根据规格自动化测试？

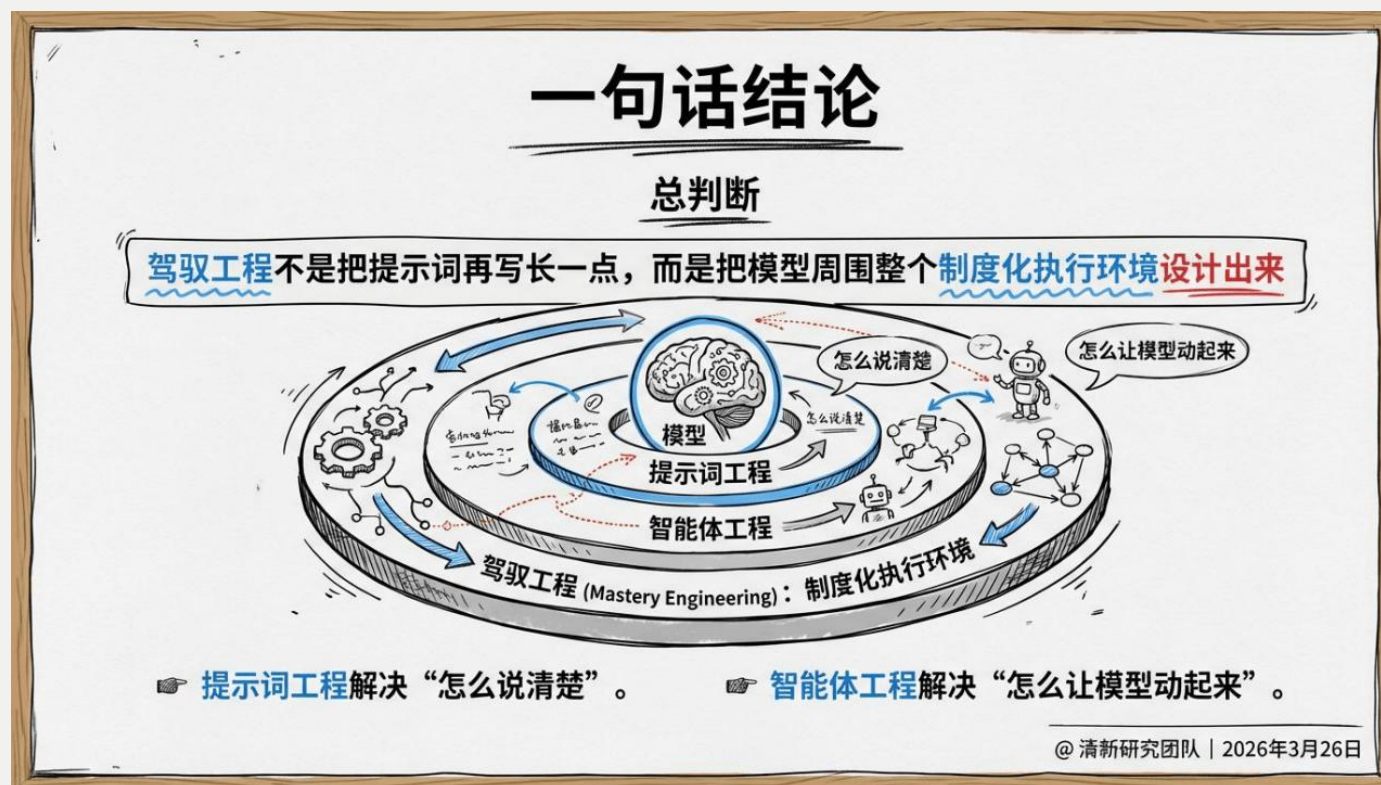
- 如何表达测试用例模版？
- 如何用测试用例生成E2E测试？

Agent as Code

终极问题：如何把上面的东西全部放到代码库中？

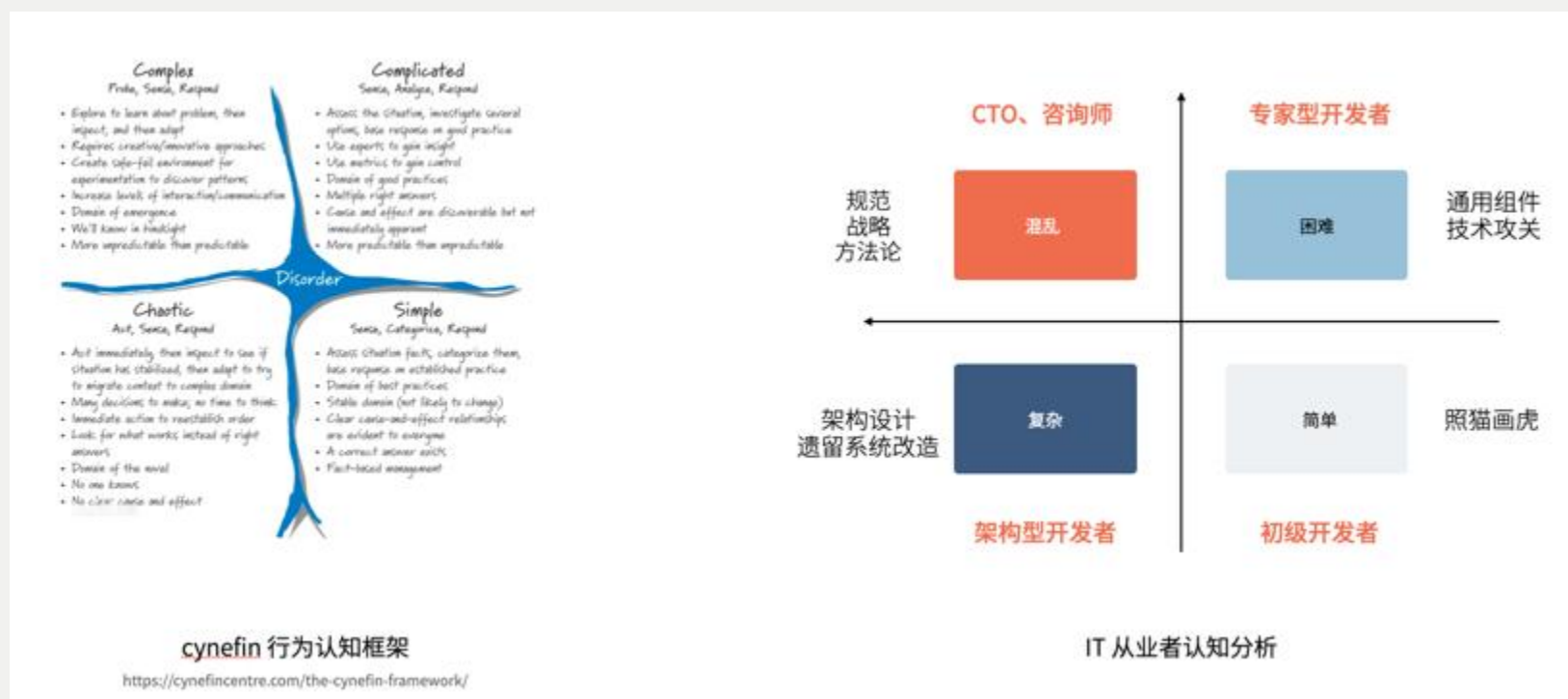
什么是 Harness ?

通俗解释 Harness: AI 友好的工作环境和执行制度

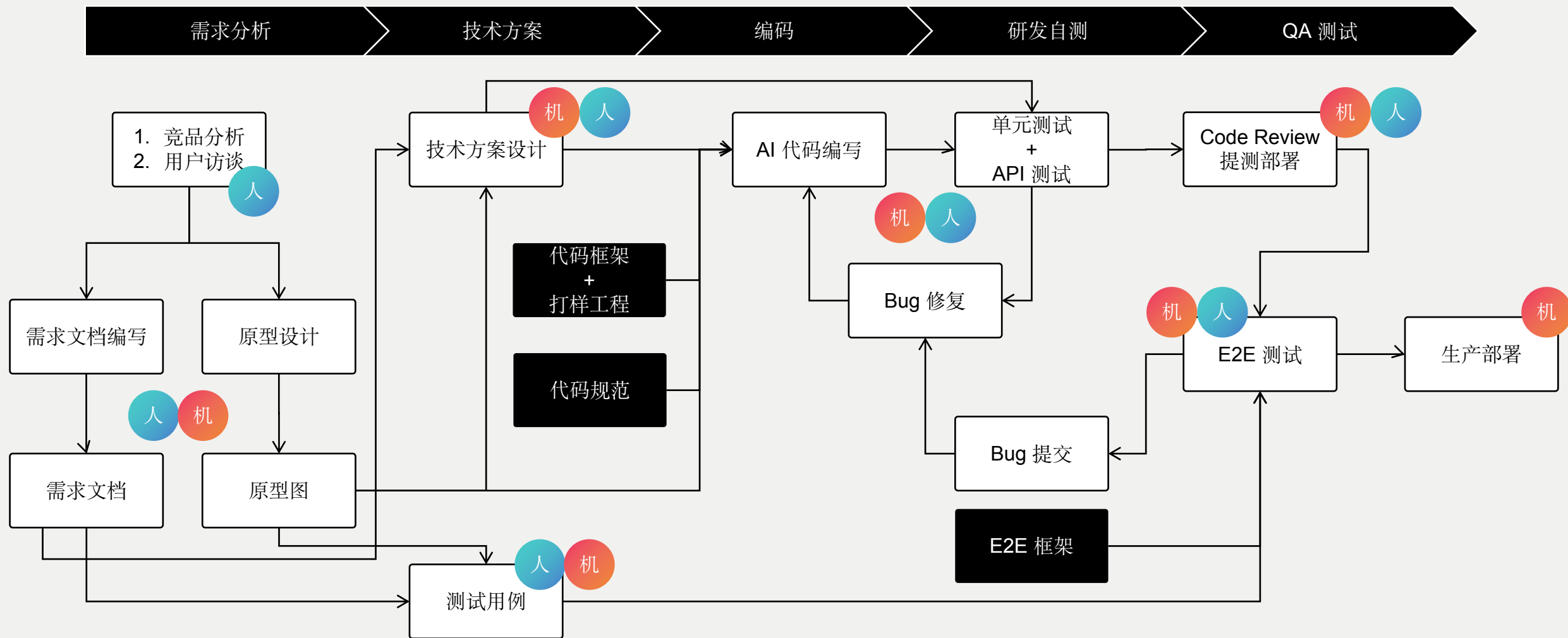


认知分级

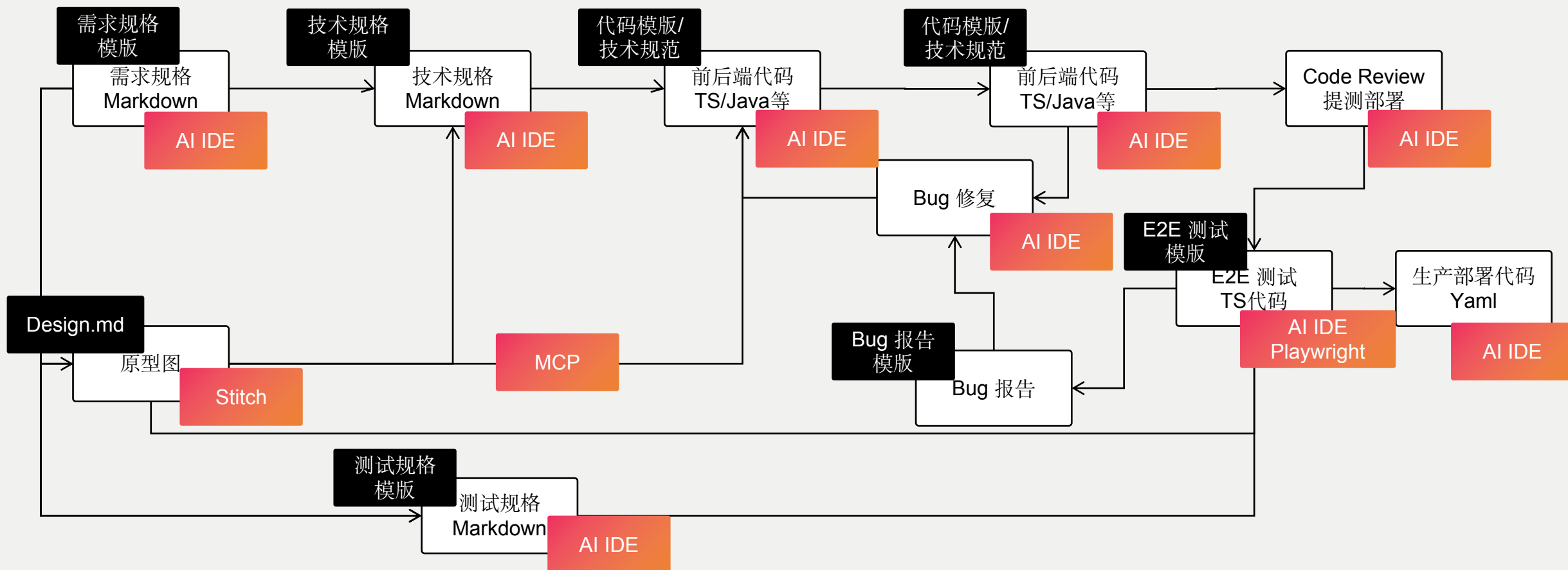
不同认知纬度的岗位被 AI 代替程度不同



AI 编程的流程和体系



产物、工具链、模版



需求衔接

如何用 Markdown 编写需求？

站在开发人员视角：什么样的需求是 AI 友好的？需求中包含哪些东西？

关键要素：业务背景+字段清单+原型链接+业务规则

如何切分模块？颗粒度多大？

一个模块一个文档。

一个模块 = 一个业务闭环 + 一套完整 CRUD + 可独立部署/测试

颗粒度判断方法：

是否有独立表结构

是否可以单独上线

如何表达字段清单？

需求中包含完整的字段清单，包括属性：字段名 | 类型 | 必填 | 默认值 | 描述 | 校验规则 |

UI展示 | 数据来源。

如何组合原型设计？

使用 figma 或者 Stitch 某个图的 URL 链接。

如何表达业务规则？

按照条目，每个规则一行，通过正交分解组合。

Markdown 需求的模版长什么样？

设定规格时，应区分新建需求和变更需求。

新建需求

新开发的场景，也叫做绿地工程，规格描述的是新建部分。

```
# 需求说明
【需求名称】：
【需求类型】：NEW
【背景】：
(为什么要做)
【目标】：
(要解决什么问题)
【范围】：
包含：
-
不包含：
-
---
# 字段表
> 所有业务数据以此为基准
| 字段名 | 中文名 | 类型 | 必填 | 描述 | 取值/来源 | 展示方式 | 创建时必填 | 可修改 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
说明：
- 类型：string / number / boolean / date / enum / object
- 取值/来源：用户输入 / 系统生成 / 枚举 / 外部数据
- 展示方式：输入框 / 下拉 / 日期 / 文本 / 隐藏
---
# 业务规则
> 所有行为必须通过规则定义
R1: 规则名称
- 适用场景：CREATE / UPDATE / COMMON
- 条件：
- 动作：
R2: 规则名称
- 适用场景：
- 条件：
- 动作：
```

变更需求

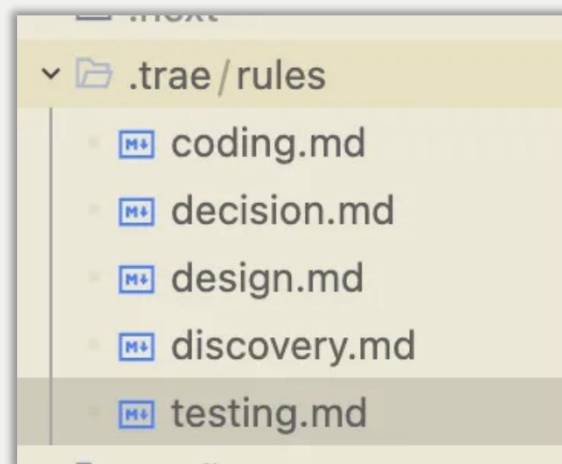
基于已有功能修改的场景，也叫做棕地工程，规格描述的是变更部分。

```
# 需求说明
【需求名称】：
【需求类型】：UPDATE
【基础版本】：
(例如 v1.2)
【目标版本】：
(例如 v1.3)
【变更前】：
(为什么要改)
【变更后】：
(改完之后的业务效果)
---
# 字段表 (变更)
> 必须使用操作标记：ADD / MODIFY / REMOVE
## [ADD]
| 字段名 | 中文名 | 类型 | 必填 | 描述 | 取值/来源 | 展示方式 | 创建时必填 | 可修改 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
---
## [MODIFY]
字段：xxx
- 原：
- 新：
字段：xxx
- 原：
- 新：
---
## [REMOVE]
字段：
- xxx
- xxx
```

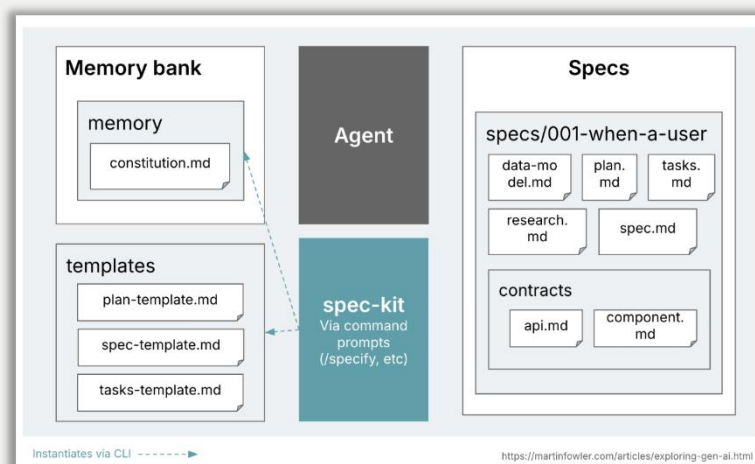

开发实现

如何让 AI 听话？

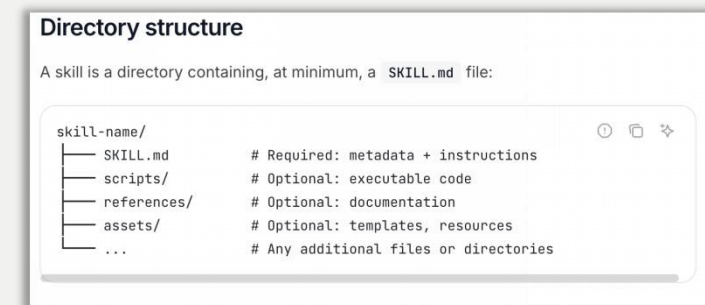
- Rules: 员工手册, 入职培训
- Specification: 任务说明
- Skills: 职业培训



用 Rules 约束当前操作
(RIPPER5)



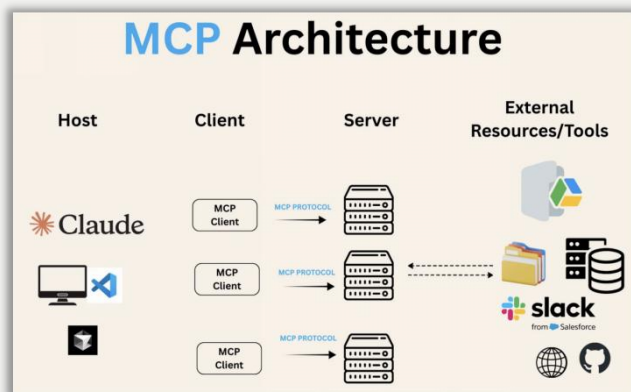
用 Specs 描述任务目标



用 Skills 描述工作方法

如何让 AI 调用外部工具？

- MCP: 外部系统交互的协议
- Skills: 把开源工具用脚本封装起来用
- Commands: 通过 cli 工具调用 外部工具



```
1 {  
2   "mcpServers": {  
3     "stitch": {  
4       "url": "https://stitch.googleapis.com/mcp",  
5       "headers": {  
6         "X-Goog-API-Key": "AQ.  
          Ab8RN6JaU3SFspX0ThEcG61Zp0TeQv-gCuarxyHazTqQUF  
          nywW"  
7       }  
8     }  
9   }  
10 }
```

常用的工具

- 数据库 MCP
- Figma MCP
- Stitch MCP
- JIRA MCP
- Commit/Push Skills

用什么 SDD 框架比较好？

Spec = 单一事实来源 (SSoT) ，甚至可以直接生成代码。其实在非 AI 时代，也是这样工作的，只不过 Spec 在很多人的“大脑”中。使用 Spec 驱动开发的模式就是 SDD。

维度	BMAD	Spec Kit	OpenSpec	Kiro
方法定位	企业级 SDD 操作系统	工程化 Spec workflow	轻量 Spec 层	IDE 原生 SDD
核心理念	Spec = 治理体系 + 多 Agent 编排	Spec = 开发入口 + Git 生命周期	Spec = 变更单元 (持续演化)	Spec = 可执行源 (代码与测试)
Spec 生命周期	全生命周期 (PRD → 架构 → 实现)	与分支绑定 (短生命周期)	长期存在 (持续演化)	持续驱动 (开发时实时参与)
Spec 粒度	大 (系统 / 模块级)	中 (Feature级)	小 (变更 / Patch级)	中 (Feature + 行为)
可执行能力说明	通过流程驱动实现，不直接运行	驱动开发流程，不参与运行	类 Prompt，可生成代码但不可验证	可生成代码、测试并自动校验
行为表达结构化	强规范	部分结构化	轻量 Given/When/Then	严格结构 (EARS)
流程控制	强 (阶段 + 审批 + Agent)	中 (Plan → Spec → Tasks)	弱 (自由演化)	强 (Spec → 生成 → 验证闭环)
Agent 参与方式	多 Agent 分工	辅助执行	Prompt 级辅助	IDE 内主导执行
自动验证能力	无 (需外部测试)	无	弱 (依赖外部)	强 (内建验证机制)
与代码关系	间接 (通过流程)	半耦合 (驱动实现)	弱耦合	强耦合 (直接生成与校验)
适用场景	大型系统 / 多团队 / 强治理	新项目 (0→1)	存量项目 / 快速迭代	小团队 / 高自动化开发
失控风险	低	中	高	中

用什么 AI 编程工具？



OpenAI

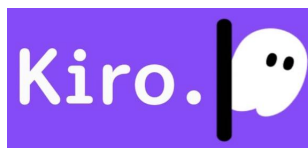
Codex



命令行类

习惯使用命令行，AI 优先的开发模式。

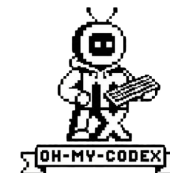
- Claude Code
- Codex CLI
- OpenCode



AI IDE

对代码掌控感更强，需要人介入程度更深。

- Cursor
- Kiro
- Trae



obra/superpowers

An agentic skills framework & software development methodology that works.

28 Contributors 154 Issues 165k Stars 14k Forks

OH MY
OPENCODE

Orchestrate curated agents. Ship faster.

增强插件

需要预制的一系列辅助工具、脚本、模版。

- Superpower
- oh-my-opencode
- oh-my-codex

一个新星：Superpower

Superpower 提供了一些内置的 **skills**，方便开箱即用，缺点是不太方便精细调整。

1. **brainstorming** - Activates before writing code. Refines rough ideas through questions, explores alternatives, presents design in sections for validation. Saves design document.
2. **using-git-worktrees** - Activates after design approval. Creates isolated workspace on new branch, runs project setup, verifies clean test baseline.
3. **writing-plans** - Activates with approved design. Breaks work into bite-sized tasks (2-5 minutes each). Every task has exact file paths, complete code, verification steps.
4. **subagent-driven-development** or **executing-plans** - Activates with plan. Dispatches fresh subagent per task with two-stage review (spec compliance, then code quality), or executes in batches with human checkpoints.
5. **test-driven-development** - Activates during implementation. Enforces RED-GREEN-REFACTOR: write failing test, watch it fail, write minimal code, watch it pass, commit. Deletes code written before tests.
6. **requesting-code-review** - Activates between tasks. Reviews against plan, reports issues by severity. Critical issues block progress.
7. **finishing-a-development-branch** - Activates when tasks complete. Verifies tests, presents options (merge/PR/keep/discard), cleans up worktree.

技术规格如何编写？

技术规格使用 DSL 驱动包含：领域模型、数据库、API、时序图、专题设计

模块	表达标准	格式	产出内容	约束 / 规范
领域模型	PlantUML/Smart Domain	.puml	实体、属性、关系 (ER / 类图)	必须表达实体关系；字段需与数据库一致；标注聚合关系
数据库	SQL DDL/DBML/Flaywa脚本	.sql	表结构、索引、约束	必须可执行；包含索引和外键；字段与领域模型一致
API	OpenAPI 3.x	.yaml / .json	接口定义、请求/响应结构	必须定义 schema；禁止只写接口说明；字段与模型一致
时序图	PlantUML	.puml	调用流程、系统交互	必须反映真实调用链；建议包含异常分支 (alt)
专题设计	Markdown	.md	架构策略（权限/事务/缓存等）	只写“策略与规则”，不重复 API/DDL；强调设计决策

```
# 技术规格文档 (Technical Specification)

## 1. 文档说明

### 1.1 背景
(描述业务背景与问题)

### 1.2 目标
* 实现能力:
* 技术边界:

---

## 2. 领域模型 (PlantUML)

```plantuml
@startuml
' 在此定义实体、属性、关系

entity User {
+id: string
+name: string
}

entity Order {
+id: string
+userId: string
}

User ||--o{ Order : places
@enduml
```

---

## 3. 数据库设计 (SQL DDL)

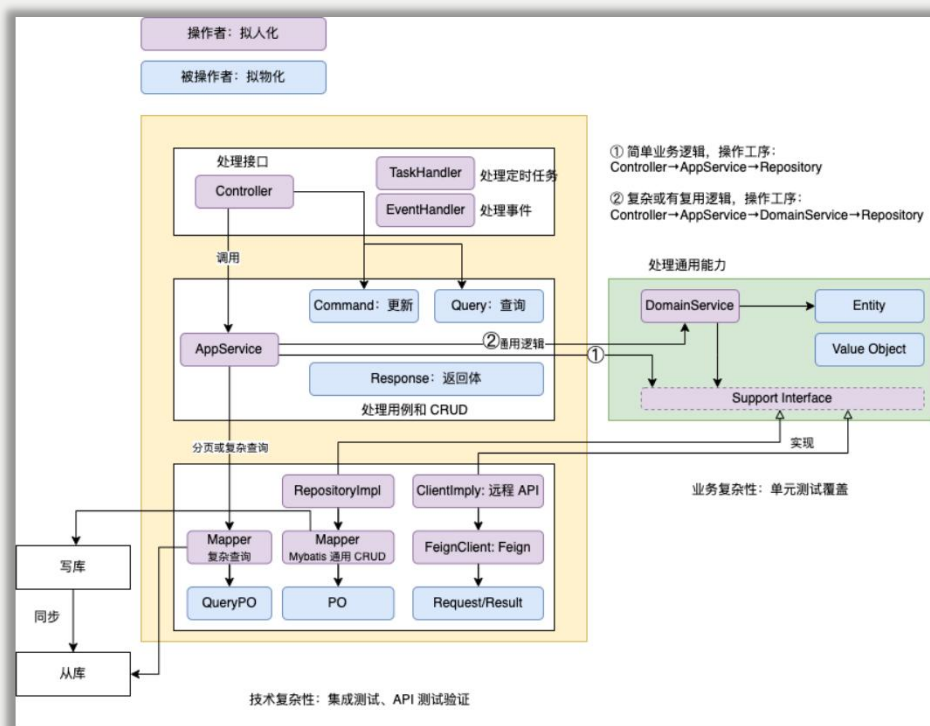
```sql
-- 在此定义表结构 (必须可执行)

CREATE TABLE user (
 id VARCHAR(64) PRIMARY KEY,
 name VARCHAR(255) NOT NULL
);

CREATE TABLE `order` (
 id VARCHAR(64) PRIMARY KEY,
 user_id VARCHAR(64) NOT NULL
);
```
```

打样工程：如何让 AI 抄作业？

原则：约定大于配置、编排逻辑和原子能力分离、操作者和被操作对象分离



<https://github.com/domain-driven-design/ddd-microservices>

如何多任务同步开发？

使用 git 的 Worktree 功能同时把一个仓库的多个分支映射到不同文件夹。

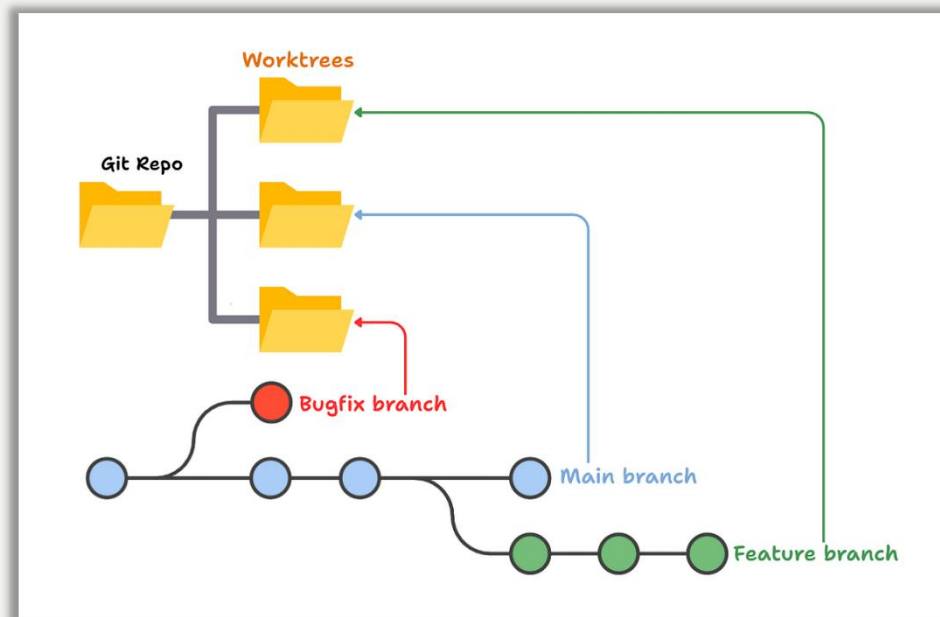
Claude Code、Codex、Cursor 等已经自动支持使用 subagent 创建 worktree，加速开发。

```
// 基于已有分支创建
git worktree add ../feature-login feature/login

// 创建新分支 + 工作区（最常用）
git worktree add -b feature-payment ../feature-payment

// 查看所有 worktree
git worktree list

// 删除工作区
git worktree remove ../feature-login
```



如何可视化技术规格用于评审？

使用 Skill 把技术规格生成 PPT 文件或者导出 PDF/HTML。

如何自动化触发代码提交、合并？

创建 Skill 完成或者通过使用 Superpower 这类插件完成。

测试验收

AI 辅助下的测试策略是什么？

| 测试类型 | 测试对象 | 工具 (含 AI) | 负责人 (人 / Agent) |
|-------------|--------------------------------|--|-----------------------------|
| Lint 代码扫描 | 代码风格、代码逻辑和片段 | TS Lint/SonarQube | 程序 |
| Code Review | 代码逻辑、架构设计、规范 (复杂度/安全/可维护性) | GitHub Copilot、Claude Code、CodeRabbit、GitHub PR | 开发者 + AI Review Agent |
| 单元测试 | 函数/类行为、边界条件、异常处理 | JUnit、pytest、Jest + AI 生成测试代码 (Copilot / Claude) | 开发者 + Test Generation Agent |
| API 测试 | 接口契约 (Schema)、输入输出、鉴权、错误码、业务规则 | Karate (主)、RestAssured (补充复杂逻辑) + AI 生成测试用例/数据 | QA + 后端开发 + API Test Agent |
| E2E 测试 | 用户关键路径 (登录/下单/支付)、跨系统流程、前后端联动 | Playwright + AI 生成脚本 / 自愈 (selector 修复、步骤补全) | QA + 自动化测试工程师 + E2E Agent |

如何自动化生成单元测试和 API 测试？

通过 Skill 或者在编码阶段通过 TDD 直接完成。

如何用测试用例生成E2E测试？

使用 Playwright + AI 生成的脚本实现。

```
import { test, expect, request } from '@playwright/test';

test.describe('User Login Flow', () => {

  test('login success and dashboard visible', async ({ page }) => {

    // 1. 打开登录页
    await page.goto('https://example.com/login');

    // 2. 填写表单
    await page.getByPlaceholder('Email').fill('test@example.com');
    await page.getByPlaceholder('Password').fill('123456');

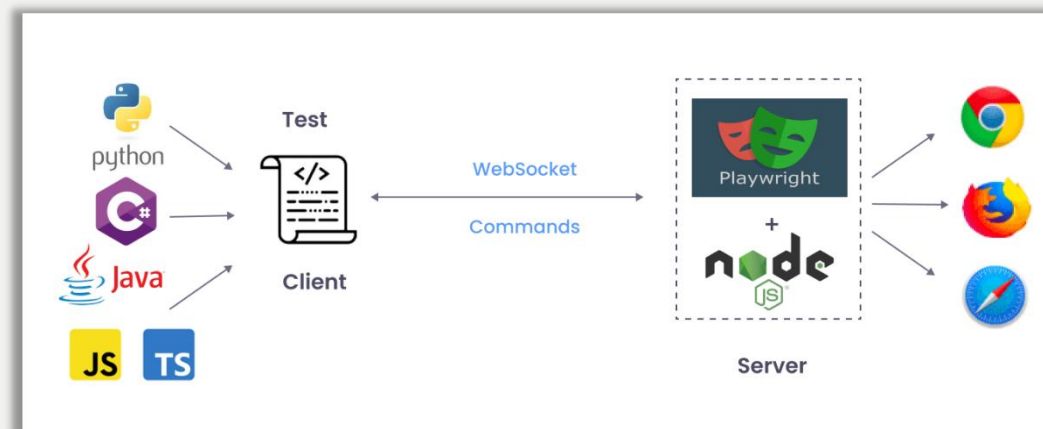
    // 3. 提交
    await page.getByRole('button', { name: 'Login' }).click();

    // 4. 等待跳转 & 断言
    await expect(page).toHaveURL(/dashboard/);
    await expect(page.getByText('Welcome')).toBeVisible();

  });

  test('login via API + set session (更稳定)', async ({ page, request }) => {
  });

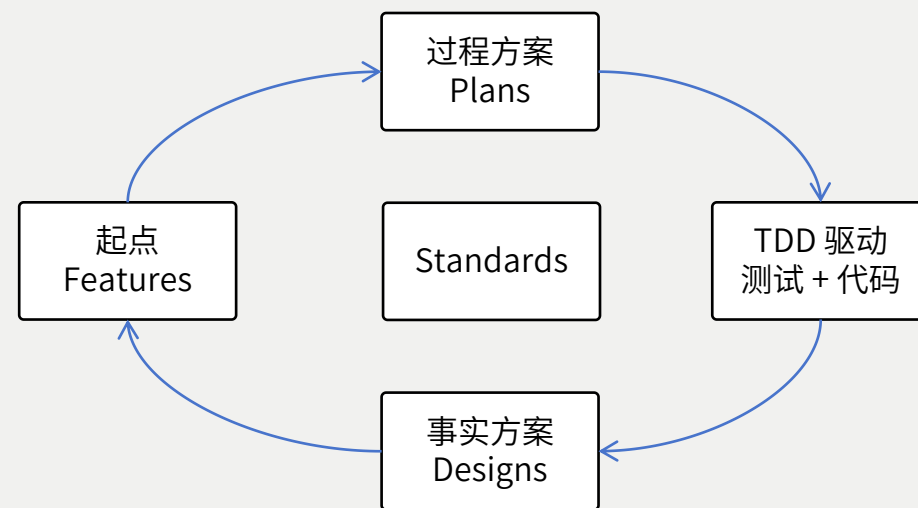
});
```



Agent as Code

SDD 文档管理策略

“法律”体系



Plan/Go 模式下的规格流转

参考资料

参考资料

- <https://openai.com/index/harness-engineering/>
- <https://martinfowler.com/articles/harness-engineering.html>
- <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- 从 Rule、Spec 到 Harness: AI Coding 的渐进式建设路径 <https://mp.weixin.qq.com/s/UCh2bPzMZjNBMBCCJysuNw>
- 让 AI 乖乖听话的几个 Rules https://mp.weixin.qq.com/s/FXNzk_y2Z2h8BVe62uEn_A
- Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- harness-engineering-in-ai-coding <https://github.com/2361485765/harness-engineering-in-ai-coding>